Timer Input Capture — Option 1(a)

ESLAB HW5 | STM32 PWM and Logic Analyzer

Objective

Measure the frequency of a PWM signal using the STM32 timer input capture function.

Setup and Method

TIM2 Channel 1 generated a 1 kHz PWM signal with 50% duty cycle on PA5 (D13). The output was connected to PB4 (D5), configured as TIM3 Channel 1 input capture. TIM3 used an 80 MHz timer clock and prescaler of 79, giving a 1 MHz counter (1 µs per count). Each rising edge triggered an interrupt, and the difference between consecutive captured counter values was used to calculate the period and frequency.

Code Implementation

1. Timer initialization and startup in main()

```
MX_TIM2_Init();
MX_TIM3_Init();
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 500);
HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_1);
```

2. Rising-edge measurement in HAL_TIM_IC_CaptureCallback()

```
uint32_t now = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
if (capture_started) {
    uint32_t diff = (now - capture_prev) & 0xFFFFU;
    if (diff > 0) {
        period_ticks = diff;
        measured_frequency = 1000000U / diff;
    }
}
capture_prev = now;
capture_started = 1;
```

The callback saves the first captured count, then uses subsequent rising edges to calculate the PWM period. The 16-bit mask handles counter wraparound; the frequency calculation assumes a 1 MHz timer counter.

Results

The STM32CubeIDE debugger showed capture_started = 1, period_ticks = 1000, and measured_frequency = 1000 Hz. Thus, the measured period was 1000 × 1 µs = 1 ms, matching the expected 1 kHz PWM setting.

Expression	Type	Value
(x)= capture_started	volatile uint8_t	1 '\001'
(x)= period_ticks	volatile uint32_t	1000
(x)= measured_frequency	volatile uint32_t	1000
Add new expression		

Figure 1. Input capture results shown in STM32CubeIDE Expressions.

Discussion

Input capture records edge timing in hardware, avoiding the timing uncertainty of software polling. Capturing two successive rising edges measures the period, but not the duty cycle; measuring HIGH time would also require a falling-edge capture. The 1 µs counter resolution limits timing precision, and counter rollover must be handled for longer periods. The observed frequency agrees with the configured PWM frequency.

Conclusion

TIM3 successfully measured the 1 kHz PWM signal generated by TIM2, demonstrating timer-based input capture.